

```

        'role_id' => $role_details['id'],
        'resource_id' => $resource_details['id'],
    );
    if ( $this->rule_exists( $resource_details['id'], $role_details['id'] ) ) {
        if ( $access == false ) {
            // Remove the rule as there is currently no need for it
            $details['access'] = ! $access;
            $this->_sql->delete( 'acl_rules', $details );
        } else {
            // Update the rule with the new access value
            $this->_sql->update( 'acl_rules', array( 'access' => $access ) );
        }
    }
    foreach( $this->rules as $key=>$rule ) {
        if ( $details['role_id'] == $rule['role_id'] && $details['resource_id'] == $rule['resource_id'] ) {
            if ( $access == false ) {
                unset( $this->rules[ $key ] );
            } else {
                $this->rules[ $key ]['access'] = $access;
            }
        }
    }
}

```

Time taking, but rewarding process

XUL

While we'll be looking into XUL (XML User Interface Language) later in this Fast Track, for now, all we need to know is that it's an XML-based grammar, which is what the Firefox user interface is written using. Think of it as HTML for UI. The developer is provided all necessary tools such as buttons, toolbars, etc., with JavaScript being used to fill in the actions. To provide functionality for our extension, we can modify the browser's UI, and integrate our add-on into the interface that way. The browser is implemented in an XUL file called `browser.xul`, archived inside `$FIREFOX_INSTALL/chrome/browser.jar`, under `/content/browser/browser.xul`.

Chrome URIs

As we know, 'chrome' refers to everything in the browser that is not the title bar or the page-view area, so essentially the entire user interface. Chrome Packages are UI bundles which can be loaded via dedicated locations in the address bar, in the form "chrome:// ____". This is useful because it creates a layer of abstraction that allows the developers to transcend the underlying architecture of the system (OS and such), which would pose a problem of consistency if hardcoded "file:// ____" links were to be used. To cut a long story short, we now have chrome URIs for all UI work. For a fun detour,